



BLUE BITS

البرمجة التفرعية

نظري (9)

ملف داعم

2026/2025

السنة الرابعة



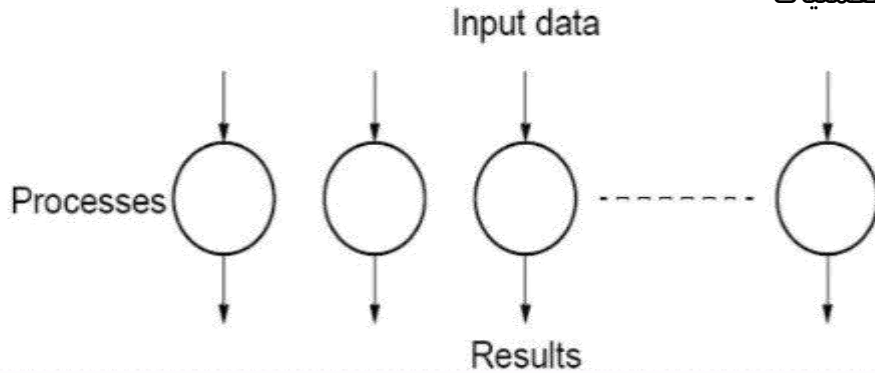
مكتبة شهاب
للشعاع النيرة - يوق يلقى على القاصي

[f](#) [@](#) [v](#) [t](#) /BlueBitsTeam

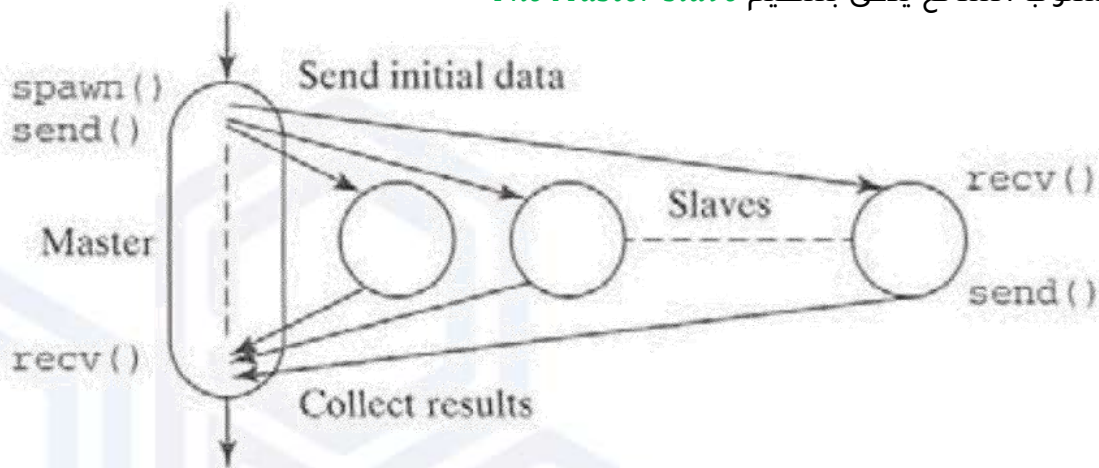
www.bluebitsteam.com

IDEAL Parallel Computation

- تتضمن البرمجة التفرعية تقسيم المشكلة إلى أجزاء منفصلة وتقوم المعالجات بحساب هذه الأجزاء.
- الحساب المتوازي المثالي هو الحساب الذي يمكن تقسيمه على الفور إلى أجزاء مستقلة تماماً يمكن تنفيذها في وقت واحد وهذا يسمى التفرع الحرج (الأمثلي)
- لا يوجد تواصل بين العمليات المنفصلة
- لا يوجد تفاعل بين العمليات



- غالباً ما تكون الأجزاء المستقلة هي حسابات متطابقة ونموذج *SPMD* مناسب
- SPMD (Single-Programmed Multiple-Data)*
- في الحساب المتوازي الحرج يجب توزيع البيانات على العمليات وجمع النتائج ودمجها بطريقة ما وهذا يعني أن العملية الواحدة يجب أن تعمل بمفردها.
- هذا الأسلوب الشائع يدعى بتنظيم *The Master slave*



هنا العلاقة فقط بين ال *Master* وال *Slave*

يمكن استخدام نهج *Master & Slave* في عملية إنشاء العمليات الثابتة (طريقة تابع *Spawn* تعتبر عملية ديناميكية).

نقوم ببساطة بوضع كل من *Master & Slave* في نفس البرنامج واستخدام عبارات *if* لتحديد إما الكود الرئيسي *Master* أو الكود التابع *Slave* بناءً على معرف العملية الخاص (*ID*).



- نستخدم هذه الطريقة في التطبيقات التي يوجد فيها الحد الأدنى من التفاعل بين العمليات التابعة. حتى لو كانت جميع العمليات التابعة متطابقة، فقد لا يوفر تعيين العمليات بشكل ثابت للمعالجات الحل الأمثل.
- ينطبق هذا بشكل خاص عندما تكون المعالجات مختلفة، كما هو الحال غالباً مع محطات العمل المتصلة بالشبكة، ومن ثم توفر تقنيات موازنة التحميل سرعة تنفيذ محسنة

التحويلات الهندسية للصورة:

- غالباً ما يتم تخزين الصور داخل جهاز الكمبيوتر بحيث يمكن تعديلها بطريقة ما
- إن صور العرض ممكن أن تكون *artificially created*
- هذا النهج عادةً مرتبط بالمصطلح *computer graphics* (رسومات الكمبيوتر)
- حيث يمكن إجراء عدد من العمليات الرسومية على الصورة المخزنة:

- 1- تحريك الصورة
 - 2- انقاص حجمها أو زيادته
 - 3- تدويرها في البعدين أو ثلاثة أبعاد
 - 4- التنعيم وكشف الحواف
- الطريقة الأساسية لتخزين صورة ثنائية الأبعاد هي الصورة البكسيلية *Pixmap*
 - يتم تخزين كل بكسل كرقم ثنائي في مصفوفة ثنائية الأبعاد
 - بالنسبة للصور بالأبيض والأسود (صورة نقطية) تكفي بت ثنائي واحد لكل بكسل
 - (1) إذا كان البكسل أبيض و (0) إذا كان البكسل *Bitmap* أسود
 - تتطلب الصور ذات التدرج الرمادي المزيد من البتات، وعادة ما يتم استخدام 8 بت لتمثيل 256 شدة لونية مختلفة أحادية اللون
 - يتطلب اللون المزيد من المواصفات. عادة ما تكون الألوان الأساسية الثلاثة هي الأحمر والأخضر والأزرق
 - *RGB* يتم تخزينها كأرقام منفصلة مكونة من 8 بت
 - يمكن استخدام ثلاث بايتات لكل بكسل: بايت واحد للأحمر وواحد للأخضر وواحد للأزرق أي 24 بت لكل
 - يمكن تقليل متطلبات تخزين الصور الملونة باستخدام جدول البحث (*lookup table*) لعقد تمثيل *RGB*.
 - على سبيل المثال بفرض أن هناك 256 لوناً مختلفاً فقط سيحتاج كل بكسل في الصورة إلى 8 بتات فقط لتحديد اللون المحدد منها عبر استخدام (*look up table*).
 - يتطلب إجراء عمليات حسابية على إحداثيات كل بكسل تحرك موضع البكسل دون التأثير على قيمته
 - بما أن التحويل على كل بكسل مستقل تماماً عن التحويلات الموجودة على وحدات البكسل الأخرى سوف نحصل على حساب تفرعي حرج حقيقي
 - نتيجة التحويل هي صورة نقطية محدثة (*bitmap*)

التحويلات الهندسية الشائعة:

1. الإزاحة *Shifting*:

يتم إعطاء إحداثيات كائن ثنائي الأبعاد مُزاح بواسطة Δx في البعد x و Δy في البعد y بواسطة:

$$X' = X + \Delta X$$



$$Y' = Y + \Delta Y$$

حيث: X', Y' هي الإحداثيات الجديدة، و X, Y هي الإحداثيات الأصلية

2. تكبير أو تصغير *Scaling*

يتم إعطاء إحداثيات كائن تم قياسه بواسطة عامل Sx في اتجاه x وعامل Sy في اتجاه y بواسطة:

$$X' = XSx$$

$$Y' = YSy$$

يتم تكبير حجم الكائن عندما يكون Sx و Sy أكبر من 1 ويتم تقليل حجمه عندما يكون Sx و Sy بين 0 و 1 ، ويبقى الكائن على حجمه عندما يكون Sx و Sy يساوي الـ 1. نلاحظ أن التكبير أو التصغير لا يلزم أن يكونا متساويين في كلا الاتجاهين x و y .

3. الدوران *Rotation*

إحداثيات جسم يدور بزاوية θ حول نقطة الأصل:

$$x' = x \cos \theta + y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

4. القص *Clipping*

يطبق هذا التحويل حدودًا مستطيلة محددة على الشكل ويحذف من الصورة المعروضة تلك النقاط الواقعة خارج المنطقة المحددة.

قد يكون هذا مفيدًا بعد تطبيق التدوير والإزاحة والقياس لإزالة الإحداثيات خارج مجال رؤية الشاشة. إذا كانت أقل قيم X و Y في المنطقة المراد عرضها هي XL و YL وأعلى قيم هي XH و YH ثم:

$$Xh \geq X' \geq Xl$$

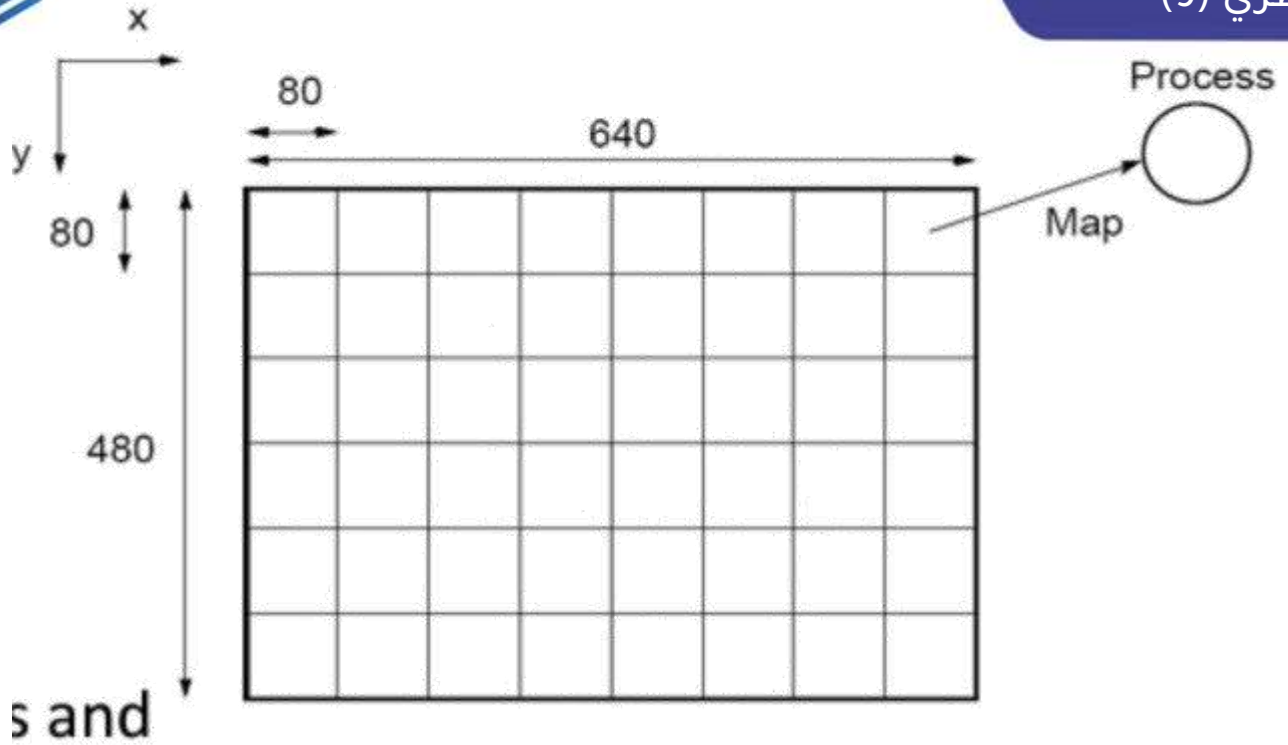
$$Yh \geq Y' \geq Yl$$

لا يجوز الخروج من مجال الصورة

القيمتين السابقتين يجب أن تتحقق ليتم عرض النقطة (x', y') وإلا لن يتم عرض هذه النقطة في التحويلات الهندسية:

- بيانات الإدخال هي الصورة النقطية التي يتم الاحتفاظ بها عادة في ملف وتنسخ إلى مصفوفة
- يمكن التلاعب بمحتويات هذه المصفوفة بسهولة دون أي تقنيات برمجية خاصة
- الاهتمام الرئيسي بالبرمجة التفرعية هو تقسيم الصورة النقطية إلى مجموعات من البكسلات لكل معالج لأنه عادة ما يكون هناك عدد من البكسلز أكبر بكثير من عدد المعالجات
- هناك طريقتان عامتان للتجميع *grouping* :
1. حسب المناطق المربعة والمستطيلة
2. حسب الأعمدة والصفوف

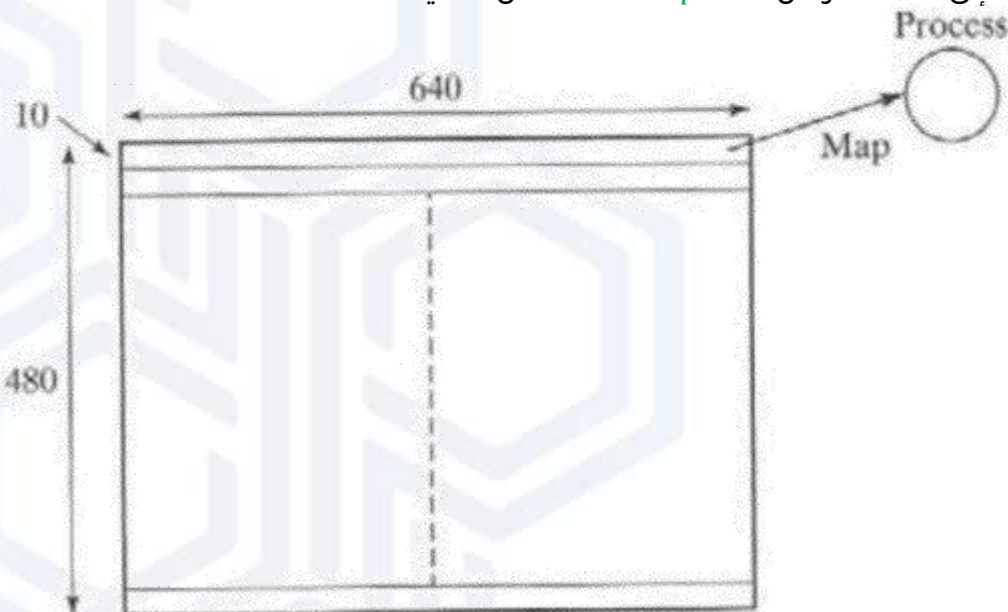
Square Regions



- يتم تعيين معالجة واحدة لمنطقة / حجرة واحدة من مصفوفة العرض الصورة عبارة عن مجموعة بكسلات 640×480 *imag* على شكل مصفوفة عدد معين من الأسطر وعدد معين من الأعمدة. هنا اقتراح المقطع 80×80 لأن حجم *data* كبير ويجب أن يكون عدد الأسطر والأعمدة يقبل القسمة على قيمة التقطيع أي لدي 48 *processes*
- 80×80 لكل *process* يعني طول المربع 80 وعرضه 80
 $640/80 = 8$ و $480/80 = 6$
 ومنه $8 \times 6 = 48$

Row Regions :

نقسم المنطقة إلى 48 سطر من 640×10 *pixels* لكل عملية





- لدي 48 processes، أي:

$$640 * 480 / 48 = 6400$$

بفرض استخدمنا عملية $master$ واحدة، و 48 عملية $slave$ والتقسيم لمجموعات يكون كل 10 أسطر بمجموعة

- نأخذ كل الأعمدة مع عدد من الأسطر مثلاً 10 ، أي كل 10 أسطر لكل الأعمدة نأخذها كشريحة ونعطيها ل $processes$

$$480 / 10 = 48 \text{ processes}$$

بما أننا أخذنا 10 أسطر بكل شريحة: $480 / 10 = 48$ معالجة $slave$ والمطلوب تقسيم المصفوفة بطريقة أعمدة وأسطر ويتم التقسيم الأفضل على الأسطر $480 / 10 = 48$ أسطر لكل معالجة

- إذا كان التحويل إزاحة، نهج $master - slave$ يبدأ عندما $Master$ يرسل رقم الصف الأول من الصفوف العشرة المراد معالجتها بواسطة كل process وعند استلام رقم الصف الخاص بها تمر كل عملية عبر كل بكسل في مجموعة الأسطر الخاصة وتحويل الإحداثيات وإرسال الإحداثيات القديمة والجديدة وإعادتها لل $master$

- يتم الاحتفاظ بالصورة النقطية في خريطة المصفوفة $map [] []$ والإعلان عن صورة نقطية مؤقتة $temp - map [] []$

- نقطة بداية ($Origin$) النظام الإحداثي للشاشة هي الزاوية اليسارية العليا

- الكود التالي لإجراء إزاحة صورة *The Pseudocode to perform an imag shift*

```

1 Master
2 for (i = 0, row = 0; i < 48; i ++, row = row + 10) /* for each process */
3 send (row, Pi); /* send row no.*/
4 for (i = 0; i < 480; i ++ ) /* initialize temp */
5 for (j = 0; j < 640; j ++ ) temp_map[i][j] = 0;
6 for (i = 0; i < (640 * 480); i ++ ) { /* for each pixel */
7 recv(oldrow, oldcol, newrow, newcol, PANY); /* accept new coords */
8 if ! ((newrow < 0) || (newrow >= 480) || (newcol < 0) || (newcol >= 640))
9 temp_map [newrow] [newcol] = map[oldrow] [oldcol];
10 }
11 for (i = 0; i < 480; i ++ ) /* update bitmap */
12 for (j = 0; j < 640; j ++ )
13 map[i][j] = temp_map[i][j];
14 Slave
15 recv(row, Pmaster); /* receive row no.*/
16 for (oldrow = row; oldrow < (row + 10); oldrow ++ )
17 for (oldcol = 0; oldcol < 640; oldcol ++ ) { /* transform coords */
18 newrow = oldrow + delta_x; /* shift in x direction */
19 newcol = oldcol + delta_y; /* shift in y direction */

```



```

20 send (oldrow, oldcol, newrow, newcol, Pmaster); /* coords to master */
21 }
22

```

تحليل التحويلات الهندسية

نفترض أن كل بكسل يتطلب خطوتين حسابيتين وأن هناك $n * n$ بكسل، إذا تم التحويلات بشكل تسلسلي سيكون هناك $n * n$ خطوة ويكون زمن التعقيد:

$$ts = 2n^2$$

تعقيد زمني متسلسل $O(n^2)$

يتكون التعقيد الزمني التفرعي من جزئين، الاتصال والحساب، كما هو الحال مع جميع الحسابات المتوازية لتمرير الرسائل.

سوف نتعامل مع الاتصالات والحساب بشكل منفصل ونجمع مكونات كل منهما لتكوين التعقيد الزمني التفرعي الشامل.

ليكن عدد العمليات p .

قبل الحساب، يجب إرسال رقم صف البداية إلى كل عملية.

في *pseudo code* الخاص بنا، نرسل أرقام الصفوف بشكل تسلسلي، وهي أرقام $send()$ p (أي p عملية إرسال)، كل منها يحتوي على عنصر بيانات واحد.

يجب على العمليات الفردية أن ترسل الإحداثيات المحولة لمجموعة وحدات البكسل الخاصة بها، الموضحة هنا مع عمليات الإرسال الفردية.

هناك عناصر بيانات $4n^2$ تم إرجاعها إلى الـ *Master*، والتي سيتعين عليه قبولها بالتسلسل.

الاتصال Communication نفترض أن المعالجات متصلة مباشرة وتبدأ بالمعادلة التالية:

$$tcomm = tstartup + m * tdata$$

$tdata$: هو الوقت الثابت لإرسال عنصر بيانات واحد وهناك m عدد الرسائل المرسل.

$tstartup$: هو الوقت الثابت لتكوين الرسالة وبدء المراسلة.

تعقيد الزمن التفرعي لوقت الاتصال هو $O(m)$

زمن الاتصال:

$$tcomm = p(tstartup + tdata) + n^2(tstartup + 4 * tdata) = O(p + n^2)$$

حيث n^2 هي أبعاد الصورة.

الحساب Computation

التنفيذ المتوازي (باستخدام مجموعات من الصفوف أو الأعمدة أو المناطق المربعة/المستطيلة) يقسم الصورة إلى مجموعات من وحدات البكسل n^2/p .

يتطلب كل بكسل إضافيتين.

يعطى حساب الزمن التفرعي بالعلاقة:

$$tcomp = 2 \left(\frac{n^2}{2} \right) = O \left(\frac{n^2}{p} \right)$$

حيث 2 عدد العمليات الحسابية



وقت التنفيذ الإجمالي *Overall execution time* :

$$tp = t_{comp} + t_{comm}$$

بالنسبة لعدد ثابت من المعالجات $O(n^2)$

"هي هكذا الأيام .. سعي دائم

إن تلتفت للخلف .. تخسر ما لديك "

2026/2025

السنة الرابعة



أعضاء الفريق

الفريق التقني

ظلال بابا	إيناس بيلوني
عائشة قلّه	تسنيم طالب
عائشة مُصطفى	حلا إبراهيم
قمر حمد	دلال تسقيه
محمد يمان جلال	ديانا صباغ
محمود غزال	زهير حقامي
وفاء حقامي	شكرية سلطان
يوسف نيّال	شهد كروم

الفريق العلمي

عائشة العُمر العلّوش	إبراهيم هبرة
عمر قظان	إسراء حاج حمدي
فاطمة الحسين	بتول لبّان
قاسم يحيى	بشرى كوكش
كريست ياغجيان	بنيامين حكيمان
محمد منير	تسنيم حوكان
مديحة جّمّال	تسنيم نور الدّين
ميرا فضلي	جميل هلال
نجم الدّين درويش	جودي جلال
هبة الله كتلو	خديجة بيازيد
وداد قلعيّة	رشا معراوي
ياسر صباغ	سدرة مبيض

هل لديك أي ملاحظة؟ لا تتردّد في مراسلتنا.

